

* Inheritance

Inheritance allows one class to acquire properties and methods from another class.

The existing class is called the parent / base / super class.

The new class is called the child / derived / subclass.

Types of Inheritance

The PPT covers:-

1. Single Inheritance.
2. Multilevel Inheritance.
3. Multiple Inheritance.
4. Hierarchical Inheritance.

* Single Inheritance

In single inheritance, one child class inherits from one parent class.

Structure :

Parent
↓
child.

Example:

</> Python

```
Class Parent:
```

```
    def show (self):  
        print ("Parent")
```

```
Class child (Parent):  
    pass.
```

```
c = child()
```

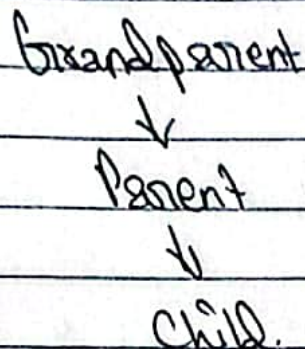
```
c.show()
```

Simple meaning:

One parent → one child.

* Multilevel Inheritance.

In multilevel inheritance, inheritance happens in levels.



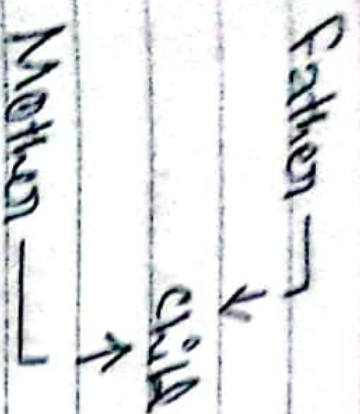
The child can access features inherited through the chain.

Simple meaning:

Parent $\rightarrow$ child $\rightarrow$ grandchild.

Multiple Inheritance.

In multiple inheritance, one child class inherits from more than one parent class.



Example:

```
<1> Python.
```

```
class Father:
    def father_skill(self):
        print("Father skill")
```

```
class Mother:
    def mother_skill(self):
        print("Mother skill")
```

```
class child(Father, Mother):
    pass
```

The child can use methods from both parents

* MRO

MRO = Method Resolution Order.

MRO defines the order in which Python searches for a method when multiple classes are involved.

It is especially important in multiple inheritance.

Python provides:

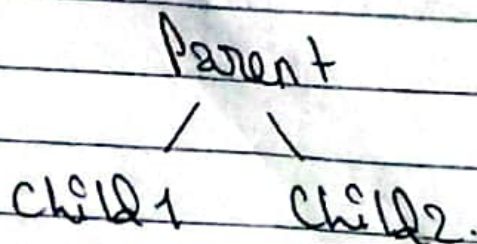
</> Python.

Class Name.mro()

to see the method Resolution Order.

Hierarchical Inheritance.

In hierarchical inheritance, one parent class has multiple child classes.



Simple meaning:

One parent $\rightarrow$ Multiple children.

Polymorphism

Polymorphism means "having more than one form."

It allows the same method or operation to behave differently in different situations.

For example, the same method name can produce different behavior depending on the object.

* Types of Polymorphism

In general OOP theory, polymorphism can be:

1. Static Polymorphism.
2. Dynamic Polymorphism.

Static Polymorphism.

Also called compile-time polymorphism.

The method call is resolved at compile time.

It includes:

- Method overloading.
- Operator overloading.

Dynamic Polymorphism.

Also called run-time polymorphism.

The behavior is determined during program execution.

* Static polymorphism in Python

Python does not traditionally support compile-time method overloading like languages such as Java or C++.

Python is dynamically typed and function behavior is resolved at runtime.

So traditional method overloading is not directly supported in Python.

11 Polymorphism in python

The PM lists two important forms:

1. Method overriding.
2. Duck typing.

Method overriding.

When a child class provides its own implementation of a method that already exists in the parent class, it is called method overriding.

Example idea:

</> Python.

```
Class Animal:
```

```
    def sound(self):
```

```
        print("Animal sound").
```

```
Class Dog (Animal):
```

```
    def sound(self):
```

```
        print("Bark").
```

Here, Dog overrides the sound() method.

* Super() keyword.

The super() keyword is used to access methods or properties from the parent class.

It is commonly used inside a child class to call the parent class's method.

Example!

<1> Python.

Class Parent:

```
def show(self):  
    print("Parent").
```

Class child (Parent):

```
def show(self):  
    super().show()  
    print("child").
```

Encapsulation.

Encapsulation means keeping data and methods together inside a class and controlling how the data is accessed.

It helps:

- Protect Data.
- Prevent accidental changes.
- Control access.
- Hide internal implementation details.

* Private Properties

In Python, a property can be made private using a double underscore prefix.

Example.

</> Python.

Class Person:

```
def __init__(self, name, age):  
    self.name = name.  
    self.__age = age.
```

Trying to access:

</> Python.

```
print(p1.__age).
```

Directly will cause an error.

* Getter method.

A getter method can be used to access a private property.

</> Python.

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name.
```

```
        self.age = age.
```

```
    def get_age(self):
```

```
        return self.age.
```

Then :

</> Python.

```
p1 = person("Tobias", 25).
```

```
print(p1.get_age()).
```

The getter provides 'controlled access to the private data.

11 Benefits of Encapsulation

1. Data Protection.

Prevents accidental modification of data.

2. Validation.

Data can be checked before it is changed.

3. Flexibility.

Internal implementation can be changed without affecting outside code.

4. Control

We can control how data is accessed and modified.

★ Protected Properties

Python uses a single underscore - as a convention for protected properties.

Example 1

</> Python.

Class Person:

```
def __init__(self, name, salary):  
    self.name = name  
    self._salary = salary
```

It can technically be accessed:

</> Python.

```
print(P1._salary)
```

but the underscore indicates that it is intended for internal/protected use.